Quantitative Finance Pricing Engines

Neel Dev Sen

July 2026

Contents

Introduction

This is a project where I develop derivative pricers using **C++** and occasionally **Python**. My goal is to do at least one a day and you can access all of my source files and header files here: **GitHub Repo**

Part I
Classical Models and Numerical Methods

1 Black-Scholes Equation

The first option that I am going to be pricing is the European option using the closed-form Black-Scholes equation.

In **GitHub** the full **C++** script is on this link: **source file**
The **header file** (first listing) is on this link: **header file**

1.1 Geometric Brownian Motion

The Black-Scholes equation takes the assumption of **Geometric Brownian Motion**

$$dS_t = rS_t dt + \sigma S_t dW_t \tag{1.1}$$

[1]
where S_t is the price of the asset at a given time t, dS_t is the infinitesimal change in the asset's price, r is the risk-free rate of interest, dt is the infinitesimal change in time, σ is the implied volatility and dW_t is the infinitesimal change in the **Wiener function**. The **Wiener function** is defined with the property:

$$W_T - W_t \sim \mathcal{N}(0, T-t) \tag{1.2}$$

where $\mathcal{N}(\mu, \sigma^{\in})$ is the **normal distribution** with mean μ and variance σ^2.[2]

1.2 Derivation of the Black-Scholes PDE

Ito's Lemma states that:

$$df(t, S_t) = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}dS_t + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}(dS_t)^2 \tag{1.3}$$

Substituting Equation 1.1 for dS_t we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left(rS_t dt + \sigma S_t dW_t\right)^2 \\ &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left(r^2 S_t^2 (dt)^2 + 2r\sigma S_t^2 (dt)(dW_t) + \sigma^2 S_t^2 (dW_t)^2\right) \end{aligned} \tag{1.4}$$

The **Ito's multiplication rules** state that:

$$(dt)^2 = 0 \tag{1.5}$$
$$(dt)(dW_t) = 0 \tag{1.6}$$
$$(dW_t)^2 = dt \tag{1.7}$$

Substituting Equations 1.5, 1.6, and 1.7 into Equation 1.4 we get:

$$\begin{aligned} df(t, S_t) &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\left(0 + 0 + \sigma^2 S_t^2 (dt)\right) \\ &= \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial S_t}\left(rS_t dt + \sigma S_t dW_t\right) + \frac{1}{2}\frac{\partial^2 f}{\partial S_t^2}\sigma^2 S_t^2 \\ &= (\frac{\partial f}{\partial t} + rS_t\frac{\partial f}{\partial S_t} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 f}{\partial S_t^2})dt + \sigma S_t\frac{\partial f}{\partial S_t}dW_t \end{aligned} \tag{1.8}$$

Define

$$V(t, S) \equiv f(t, S_t) \tag{1.9}$$

Using the result from Equation 1.8

$$dV(t, S) = (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 V}{\partial S^2})dt + \sigma S\frac{\partial V}{\partial S}dW_t \tag{1.10}$$

Now we will define a delta-hedged portfolio which means a portfolio with an option V as well as shorting Δ shares with value S.
Δ is an **option Greek** which is the partial derivative of the value of an option with respect to the underlying asset price $\Delta = \frac{\partial V}{\partial S}$
The value of the portfolio Π is shown through the equation below

$$\Pi = V - \Delta S \tag{1.11}$$

For an infinitesimal time step this becomes:

$$d\Pi = dV - \Delta dS \tag{1.12}$$

Substituting Equation 1.10 for dV and Equation 1.1 for dS we get.

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 V}{\partial S^2})dt + \sigma S\frac{\partial V}{\partial S}dW_t - \Delta(rSdt + \sigma SdW_t) \\ &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S_t^2\frac{\partial^2 V}{\partial S^2} - \Delta rS)dt + (\sigma S\frac{\partial V}{\partial S} - \Delta\sigma S)dW_t \end{aligned} \tag{1.13}$$

Since $\Delta = \frac{\partial V}{\partial S}$, Equation 1.13 becomes:

$$\begin{aligned} d\Pi &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rS\frac{\partial V}{\partial S})dt + (\sigma S\frac{\partial V}{\partial S} - \sigma S\frac{\partial V}{\partial S})dW_t \\ &= (\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} - rS\frac{\partial V}{\partial S})dt \\ &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2})dt \end{aligned} \tag{1.14}$$

The **no-arbitrage condition** in a **risk-free portfolio** is:

$$d\Pi = r\Pi dt \tag{1.15}$$

Substituting this into Equation 1.14 we get:

$$\begin{aligned} r\Pi dt &= (\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2})dt \\ r\Pi &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \end{aligned} \tag{1.16}$$

Substituting Equation 1.11 into Equation 1.16 we get:

$$\begin{aligned} r(V - \Delta S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \\ r(V - \frac{\partial V}{\partial S}S) &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \\ rV - rS\frac{\partial V}{\partial S} &= \frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \end{aligned} \tag{1.17}$$

This gives us the final partial differential equation which is also the **Black-Scholes equation**:

$$\boxed{\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0} \tag{1.18}$$

In this equation V is the option's price, t is the current time.
$\frac{\partial V}{\partial t}$ is the rate of change of the option's price to time, $\frac{\partial V}{\partial S}$ is the rate of change of the option's price to the underlying stock price and $\frac{\partial^2 V}{\partial S^2}$ is a concavity factor on how the option price changes with the stock price. The Black-Scholes can be solved for a call option and a put option. [3].

1.3 Closed-Form Solution for European Call Options

The closed-form solution for the Black-Scholes equation where the option is a call option is:

$$\begin{aligned} C(t,S) &= S\Phi(d_1) - Ke^{-r(T-t)}\Phi(d_2) \\ d_1 &= \frac{\ln(\frac{S}{K}) + (r + \frac{1}{2}\sigma^2)(T-t)}{\sigma\sqrt{T-t}} \\ d_2 &= d_1 - \sigma\sqrt{T-t} \end{aligned} \tag{1.19}$$

In this equation $T-t$ represents the time to maturity and K represents the strike price. [3].

$\Phi(z)$ is the standard normal cumulative distribution function which can be represented with the error function $\text{erf}(z)$:

$$\Phi(z) = \frac{1}{2}(1 + \text{erf}(\frac{z}{\sqrt{2}})) \tag{1.20}$$

In C++ this can be implemented as:

```cpp
#include <cmath>
#include <type_traits>

template <typename T>
auto normalCDF(T x) -> std::common_type_t<double, T>
{
    using commonType = std::common_type_t<double, T>;

    return static_cast<commonType>(0.5) * (static_cast<commonType>(1) +
    ↪  std::erf(x / std::sqrt(2.0)));
}
```

Listing 1.1: Normal cumulative distribution function

This function is defined in a header file called **functions.hpp** and the mechanism of this function is basically to return the result from Equation 1.19. Using the **cmath** header I compute the error function using **std::erf()**. This function returns a value with at least a type **double** depending on what type **x** is.

The next listing is a **struct** that is used throughout each engine that stores all of the data required to price a **Black Scholes option**. This struct is also in a header file called **black-scholes-struct.hpp**.

```
1   #ifndef GBMOPTIONS
2   #define GBMOPTIONS
3   #include <type_traits>
4
5   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
6   struct OptionDataBS
7   {
8       using CommonType = std::common_type_t <double, S_t, K_t, r_t,
        ↪  sigma_t, T_t>;
9       CommonType spot {};
10      CommonType strike {};
11      CommonType rate {};
12      CommonType volatility {};
13      CommonType maturity {};
14  };
15
16  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
17  OptionDataBS(S_t, K_t, r_t, sigma_t, T_t) -> OptionDataBS<S_t, K_t,
    ↪  r_t, sigma_t, T_t>;
18  #endif
```

Listing 1.2: Black Scholes Option Data Struct

Lines 1,2 and 18 are all part of a **header guard**. The struct **OptionDataBS** uses a template giving the opportunity for all of the data to have unique data types when passed into the struct. Line 8 creates a **type alias** which takes the common type of all of these parameters and sets it to something of at least type double. These types are then used to initiate the **spot, strike, rate, volatility** and **maturity** of this option. Line 16-17 is just a **class argument template deduction guide** so the compiler knows how to deduce the types of the arguments passed

Now to implement the function shown in Equation 1.19 in C++ we will use both the function and struct defined above:

```
1  #include <cmath>
2  #include <type_traits>
3  #include "../Headers/functions.hpp"
4  #include "../Headers/black_scholes_struct.hpp"
5
6  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
   ↪  typename T_t>
7  auto blackScholesCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
   ↪  data) -> decltype(data.spot)
8  {
9      using CommonType = decltype(data.spot);
10     auto sqrt_maturity {std::sqrt(data.maturity)};
11
12     auto d1 {(std::log(data.spot / data.strike) + (data.rate +
       ↪  (static_cast<CommonType>(0.5) * data.volatility *
       ↪  data.volatility)) * (data.maturity))/ (data.volatility *
       ↪  sqrt_maturity)};
13     auto d2 {d1 - data.volatility * sqrt_maturity};
14
15     auto C {data.spot * normalCDF(d1) - data.strike *
       ↪  std::exp(-data.rate * data.maturity) * normalCDF(d2)};
16     return C;
17 }
```

Listing 1.3: Black Scholes Call using Closed Form Solution

The function in this listing is called **blackScholesCall**. This listing passes a reference to the struct **OptionDataBS** called **data** (since copying a struct is expensive) The return type of this function is deduced to be **commonType** which is also teh type of **data.spot** (hence I used **decltype**). The formula for d_1 is shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in Equation 1.19 is shown in line 15 using the previously defined function **normalCDF**. The function returns the value of the **call option** with at least type **double**.

1.4 Time Complexity of the Black-Scholes Formula

Since the Black-Scholes Formula is a closed form solution, no iterations occur and the time complexity is simply:

$$\boxed{\mathcal{O}(1)} \tag{1.21}$$

1.5 Closed-Form Solution for European Put Options

The closed-form solution for the Black-Scholes equation fo ra put option is:

$$P(t, S) = Ke^{-r(T-t)}\Phi(-d_2) - S\Phi(-d_1) \tag{1.22}$$

using the same definitions of d_1 and d_2 in Equation 1.19 [3]

The implementation of this in C++ is:

```
1   #include <cmath>
2   #include <type_traits>
3   #include "../Headers/functions.hpp"
4   #include "../Headers/black_scholes_struct.hpp"
5
6   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
7   auto blackScholesPut(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
    ↪  data) -> decltype(data.spot)
8   {
9       using CommonType = decltype(data.spot);
10      auto sqrt_maturity {std::sqrt(data.maturity)};
11
12      auto d1 {(std::log(data.spot / data.strike) + (data.rate +
        ↪  (static_cast<CommonType>(0.5) * data.volatility *
        ↪  data.volatility)) * (data.maturity)) / (data.volatility *
        ↪  sqrt_maturity)};
13      auto d2 {d1 - data.volatility * sqrt_maturity};
14
15      auto P  {data.strike * std::exp(-data.rate * data.maturity) *
        ↪  normalCDF(-d2) - data.spot * normalCDF(-d1)};
16      return P;
17  }
18
```

Listing 1.4: Black Scholes Put using Closed Form Solution

The function in this listing is called **blackScholesPut**. This listing is almost identical to the one for **blackScholesCall**. The formula for d_1 is again shown in line 12 and the formula for d_2 is shown in line 13. Finally the result in Equation 1.22 is computed in line 15 (stored as **P**) using the function **normalCDF**. The function returns the value of the **put option** with at least type **double**.

2 Monte Carlo Simulations

The source file can be found on my **GitHub**

2.1 Derivation of the value of a Stock

First we will use the **Geometric Brownian Motion** assumption defined in Equation 1.1

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t$$
$$\frac{dS_t}{S_t} = r\,dt + \sigma\,dW_t$$

Define:

$$X_t = \ln(S_t) \tag{2.1}$$

Using **Ito's Lemma** we can represent dX_t in terms of dS_t like this:

$$dX_t = \frac{dX_t}{dS_t}\,dS_t + \frac{1}{2}\frac{d^2X_t}{dS_t^2}(dS_t)^2$$

Since $\frac{dX_t}{dS_t} = \frac{1}{S_t}$ and $\frac{d^2X_t}{dS_t^2} = -\frac{1}{(S_t)^2}$. Substituting this in the previous equation we get:

$$\begin{aligned} dX_t &= \frac{1}{S_t}\left(rS_t\,dt + \sigma S_t\,dW_t\right) - \frac{1}{2S_t^2}\left(rS_t\,dt + \sigma S_t\,dW_t\right)^2 \\ dX_t &= r\,dt + \sigma\,dW_t - \frac{1}{2S_t^2}\left(r^2S_t^2(dt)^2 + 2r\sigma S_t^2\,dt\,dW_t + \sigma^2S_t^2(dW_t)^2\right) \end{aligned} \tag{2.2}$$

Using **Ito's multiplication rules** defined in Equation 1.7, we can simplify the equation to:

$$\begin{aligned} dX_t &= r\,dt + \sigma\,dW_t - \frac{1}{2S_t^2}\left(\sigma^2S_t^2\,dt\right) \\ dX_t &= \left(r - \frac{1}{2}\sigma^2\right)dt + \sigma\,dW_t \end{aligned} \tag{2.3}$$

Introducing a dummy variable s we can finally integrate all of the terms:

$$\begin{aligned} \int_t^T dX_s &= \int_t^T \left(r - \frac{1}{2}\sigma^2\right)ds + \int_t^T \sigma\,dW_s \\ X_T - X_t &= \left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \end{aligned} \tag{2.4}$$

Substituting $ln(S_T) = X_T$ and $ln(S_t) = X_t$ into Equation 2.4 we get:

$$\begin{aligned} \ln(S_T) - \ln(S_t) &= \left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \\ \ln\left(\frac{S_T}{S_t}\right) &= \left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t) \\ \frac{S_T}{S_t} &= \exp\left(\left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t)\right) \end{aligned} \tag{2.5}$$

This leaves us with the equation of a stock which is:

$$\boxed{S_T = S_t \exp\left(\left(r - \frac{1}{2}\sigma^2\right)(T-t) + \sigma(W_T - W_t)\right)} \tag{2.6}$$

[4]

2.2 Monte Carlo Simulations for BS Call Options

In order to price an option with the Monte Carlo Simulation we must use the call payoff definition of an option. A payoff for a call is defined as:

$$\text{Payoff} = \max(S_T - K, 0) \tag{2.7}$$

Where S_T is the stock price at the time of maturity T

When we use a Monte Carlo Simulation we simulate a lot of these values for S_T to find the expected value of a payoff. After we find the expected value of a payoff, we must discount it using the risk-free rate r like this:

$$\boxed{C = e^{-r(T-t)}\mathbb{E}(\max(S_T - K, 0))} \tag{2.8}$$

[4] where $e^{-r(T-t)}$ is the discount factor and S_T is the same as defined in Equation 2.6 In order to simulate the Wiener function, we use Equation 1.2

```cpp
#include <cmath>
#include <random>
#include <algorithm>
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t, typename T_t>
auto monteCarloBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>& data, int simulations) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);
    std::random_device rd{};
    std::mt19937 mt{rd()};
    std::normal_distribution W {static_cast<commonType>(0), std::sqrt(data.maturity)};
    commonType payoffs {};

    for (int i {}; i < simulations; ++i)
    {
        commonType S_final {data.spot * std::exp((data.rate - 0.5 * data.volatility * data.volatility) * data.maturity + (data.volatility * W(mt)))};
        payoffs += std::max(S_final - data.strike, static_cast<commonType>(0));
    }
    payoffs /= simulations;

    return std::exp(-data.rate * data.maturity) * payoffs;
}
```

Listing 2.1: Black-Scholes Call using Monte Carlo Simulations

First, this function takes in the struct defined in Listing 1.2 as well as another parameter called **simulations**.
In line 10 we define a random device in order to seed our random number generator. For the random number generator in line 11 we use a **Mersenne Twister** generator. Line 12 uses the **std::normal distribution** function from the **cmath** header that will normalise the random variable we generate into a standard normal random variable as shown in Equation 1.2. In line 15 a **for loop** is made with **simulation** number of iterations.
Line 17 it computes Equation 2.6 and then is used in the Equation 2.7 which is computed in line 18.
After the for loop ends, an average of the payoffs is taken in line 20 by dividing the

total payoffs by the number of simulations, and finally in line 22 this average payoff is discounted and then returned as the call price.

2.3 Time Complexity of Monte Carlo Simulations

Let N be the number of simulations in the Monte Carlo Simulation. There are only N iterations each time a Monte-Carlo simulation is used, which makes the time complexity expression simply

$$\boxed{\mathcal{O}(N)} \tag{2.9}$$

2.4 Monte Carlo Simulations for BS Put Options

For a put option, it is pretty much the opposite, the payoff for a put option is defined as:

$$\text{Payoff} = \max(K - S_T, 0) \tag{2.10}$$

The equation for the put option value at time t is identical to that of a call option, just the payoff is different. The value of a put option is represented as:

$$\boxed{P = e^{-r(T-t)}\mathbb{E}(\max(K - S_T, 0))} \tag{2.11}$$

[4] where $e^{-r(T-t)}$ is the discount factor and S_T is the same as defined in Equation 2.6

```
1  #include <cmath>
2  #include <random>
3  #include <algorithm>
4  #include "../Headers/black_scholes_struct.hpp"
5
6  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
   ↪  typename T_t>
7  auto monteCarloBSPut(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
   ↪  data, int simulations) -> decltype(data.spot)
8  {
9      using commonType = decltype(data.spot);
10     std::random_device rd{};
11     std::mt19937 mt{rd()};
12     std::normal_distribution W {static_cast<commonType>(0),
       ↪  std::sqrt(data.maturity)};
13     commonType payoffs {};
14
15     for (int i {}; i < simulations; ++i)
16     {
17         commonType S_final {data.spot * std::exp((data.rate - 0.5 *
           ↪  data.volatility * data.volatility) * data.maturity +
           ↪  (data.volatility * W(mt)))};
18         payoffs += std::max(data.strike - S_final ,
           ↪  static_cast<commonType>(0));
19     }
20     payoffs /= simulations;
21
22     return std::exp(-data.rate * data.maturity) * payoffs;
23 }
```

Listing 2.2: Black-Scholes Put using Monte Carlo Simulations

This implementation is almost identical except in line 18 the payoffs evaluate Equation 2.10 instead of Equation 2.7

3 Binomial Trees

All the files on my **GitHub** are listed below:
3.1-binomial-tree-BS.cpp
3.4-binomial-tree-alternative.cpp

3.1 Deriving the Risk-Neutral Probability

A Binomial Tree is a numerical method that is produced by creating values until the time to maturity. The nodes at maturity are all the payoffs, and with the binomial tree we work backwards to find the price of the option at the initial time.

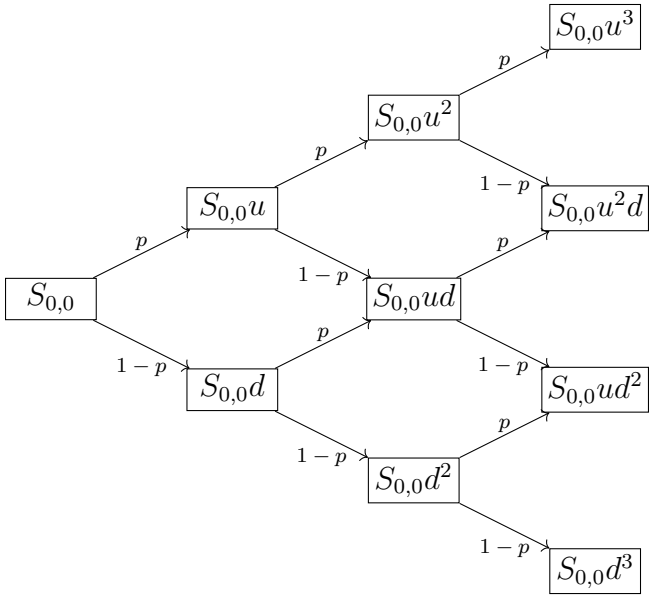


Figure 1: Binomial Tree for the underlying asset price

[5] This is what a binomial tree for a stock looks like. It can go up with probability p, and go down with probability $1-p$. The stock $S_{i,j}$ is the node that is at step i and up j nodes

$$S_{i,j} = u^j d^{i-j} S_{0,0} \tag{3.1}$$

In order to derive the values for u and d we are going to use the equation of a stock

from Equation 2.6

$$S_T = S_t \exp((r - \frac{1}{2}\sigma^2)(T-t) + \sigma(W_T - W_t))$$

On an interval of time Δt we have

$$\frac{S_{t+\Delta t}}{S_t} = \frac{S_t \exp((r - \frac{1}{2}\sigma^2)(\Delta t) + \sigma(W_{t+\Delta t} - W_t))}{S_t}$$
$$\frac{S_{t+\Delta t}}{S_t} = \exp((r - \frac{1}{2}\sigma^2)(\Delta t) + \sigma(W_{t+\Delta t} - W_t))$$

Using equation 1.2 we have:

$$W_{t+\Delta t} - W_t \sim \mathcal{N}(0, \Delta t)$$
$$W_{t+\Delta t} - W_t \sim \Delta t(\mathcal{N}(0,1))$$

Letting $Z \sim \mathcal{N}(0,1)$ we have:

$$\frac{S_{t+\Delta t}}{S_t} = \exp((r - \frac{1}{2}\sigma^2)(\Delta t) + \sigma\sqrt{\Delta t}Z$$

At very small Δt , $\sqrt{\Delta t} \gg \Delta t$ which means this equation can be approximated to:

$$\frac{S_{t+\Delta t}}{S_t} \approxeq \exp(\sigma\sqrt{\Delta t}Z) \tag{3.2}$$

Now we define a Binomial Normal Variable Z_{bin} such that $\Pr(Z_{bin} = 1) = \frac{1}{2}$ and $\Pr(Z_{bin} = -1) = \frac{1}{2}$ This has the same expected value as the Wiener Function:

$$\mathbb{E}[Z_{bin}] = (\frac{1}{2} \times 1) + (\frac{1}{2} \times -1) = 0 \tag{3.3}$$

And also the same variance as the Wiener Function:

$$\begin{aligned} \mathrm{Var}(Z_{bin}) &= \mathbb{E}[Z_{bin}^2] - (\mathbb{E}[Z_{bin}])^2 \\ \mathrm{Var}(Z_{bin}) &= (\frac{1}{2} \times (1)^2) + (\frac{1}{2} \times (-1)^2) - 0^2 \\ \mathrm{Var}(Z_{bin}) &= 1 \end{aligned} \tag{3.4}$$

We define the up factor u as the factor when $Z_{bin} = 1$ and down factor d when $Z_{bin} = -1$

$$u = \frac{S_{t+\Delta t}}{S_t}$$
$$u = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Substituting $Z_{bin} = 1$ we get the value of u to be:

$$\boxed{u = e^{\sigma\sqrt{\Delta t}}} \tag{3.5}$$

[5] Doing the same for the down factor:

$$d = \frac{S_{t+\Delta t}}{S_t}$$
$$d = e^{\sigma\sqrt{\Delta t}Z_{bin}}$$

Now substituting $Z_{bin} = -1$ we get the value of d to be:

$$\boxed{d = e^{-\sigma\sqrt{\Delta t}}} \tag{3.6}$$

[5] When deriving the risk-neutral probability we know that the expected value of a stock at time $t + \Delta t$ is the value of that of t compounded for the time Δt or:

$$\mathbb{E}[S_{t+\Delta t}] = S_t e^{r\Delta t} \tag{3.7}$$

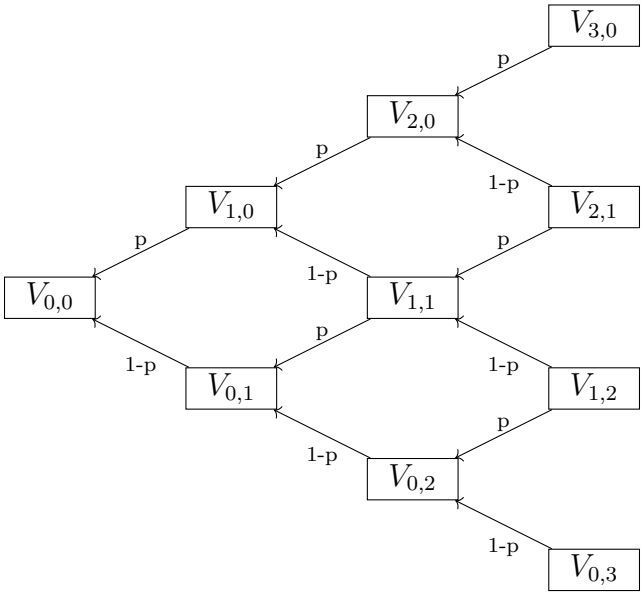
According to the binomial tree

$$\begin{aligned}
\mathbb{E}[S_{t+\Delta t}] &= S_t(up + (1-p)d) \\
S_t e^{r\Delta t} &= S_t(up + (1-p)d) \\
e^{r\Delta t} &= up + (1-p)d \\
e^{r\Delta t} &= up + d - dp \\
e^{r\Delta t} &= (u-d)p + d \\
\frac{e^{r\Delta t} - d}{u-d} &= p
\end{aligned}$$

Substituting Equations 3.5 and 3.6 for u and d respectively we get the risk neutral probability p to be:

$$\boxed{p = \frac{e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}} \tag{3.8}$$

[5]



each step $= \Delta t$

Figure 2: Reverse Binomial Tree to find payoffs

[6] For the Binomial Tree, we first compute the terminal payoffs. Using the terminal payoffs we work in a reverse fashion in order to calculate the payoffs in the nodes before, with this formula:

$$V_{i,j} = e^{-r\Delta t}(pV_{i+1,j} + (1-p)V_{i+1,j}) \tag{3.9}$$

[5] In order to create the terminal payoffs for a Binomial Tree of size n, the formula for a European Call is:

$$V_{n,k} = \max(u^k d^{n-k} S_{0,0} - K, 0) \tag{3.10}$$

[5] where $0 \leq k \leq n$
For a put European Put option, the terminal payoffs are:

$$V_{n,k} = \max(K - u^k d^{n-k} S_{0,0}, 0) \tag{3.11}$$

[5]

3.2 Binomial Tree for BS Call Options

```cpp
#include <cmath>
#include <vector>
#include <type_traits>
#include <algorithm>
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
auto binomialTreeBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t,
↪  T_t>& data, const int N) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);

    auto sqrt_deltat {std::sqrt(data.maturity /
    ↪  static_cast<commonType>(N))};
    auto u {std::exp(data.volatility * sqrt_deltat)};
    auto d {std::exp(-data.volatility * sqrt_deltat)};
    auto p {(std::exp(data.rate * data.maturity / N) - d) / (u -d)};

    std::vector<commonType> V_current (N + 1);
    for (int j {}; j < N + 1; ++j)
    {
        V_current[j] = std::max(data.spot *
        ↪  std::pow(static_cast<double>(u) , static_cast<double>(j)) *
        ↪  std::pow(static_cast<double>(d), static_cast<double>(N-j))
        ↪  - data.strike , static_cast<commonType>(0));
    }

    for (int i {}; i < N ; ++i)
    {
        std::vector<commonType> V_new (N-i);
        for (int k {}; k < N-i; ++k)
        {
            V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
        }
        std::swap(V_current, V_new);
    }

    return V_current[0] * std::exp(-data.rate * data.maturity);
}
```

Listing 3.1: Black-Scholes-Merton Put using Monte Carlo Simulations

This function **binomialTreeBSCall** passes in the struct **OptionDataBS** and another integer parameter **N** which represents the total computes Equation 3.5 in line 13, Equation 3.6 in line 14 and Equation 3.8 in line 15.
Lines 17-20 basically is evaluating Equation 3.10 and storing it in a vector of size $N + 1$. This is then used at the start of line 23.
The mechanism here is that it first initialises a new vector that represents the column of nodes at the previous step.
Line 28 evaluates Equation 3.9 without the discount factor $e^{-r\Delta t}$. In this program $\mathrm{V}_{\mathrm{current}}[k+1]$ is the up node and $\mathrm{V}_{\mathrm{current}}[k]$ is the down node. After it is done evaluating that, in line 30 it swaps the vectors, so we can use it again.
The loop variable **i** is incremented up a value, and a new vector $\mathrm{V}_{\mathrm{new}}$ is defined with a length of $N - i$. This continues untill $i = N - 1$ when the loop stop since the vector$\mathrm{V}_{\mathrm{current}}$ has a length of only 1 value. This is the final terminal payoff. Finally in line 33 this value is returned, as well as discounted with a factor of $e^{-r(T-t)}$.

3.3 Time Complexity of Binomial Trees

The time complexity of this algorithm mainly comes from the double nested loops from lines 23-31. The total number of iterations in the program is:

$$\begin{aligned} N_{iterations} &= N + \sum_{i=0}^{N-1}(N-i) \\ N_{iterations} &= N + N^2 - \frac{N(N-1)}{2} \\ N_{iterations} &= \frac{N^2+3N}{2} \end{aligned} \tag{3.12}$$

Substituting this into the time complexity expression:

$$\mathcal{O}(\frac{N^2+3N}{2})$$

Since we take the term with the highest order and exclude constants, the final time complexity of the Binomial Tree is:

$$\boxed{\mathcal{O}(N^2)} \tag{3.13}$$

3.4 Binomial Tree for BS Put Options

```cpp
#include <cmath>
#include <vector>
#include <type_traits>
#include <algorithm>
#include "../Headers/black76_structs.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
auto binomialTreeBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t,
↪  T_t>& data, const int N) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);

    auto sqrt_deltat {std::sqrt(data.maturity /
    ↪  static_cast<commonType>(N))};
    auto u {std::exp(data.volatility * sqrt_deltat)};
    auto d {std::exp(-data.volatility * sqrt_deltat)};
    auto p {(std::exp(data.rate * data.maturity / N) - d) / (u -d)};

    std::vector<commonType> V_current (N + 1);
    for (int j {}; j < N + 1; ++j)
    {
        V_current[j] = std::max(data.strike - data.spot *
        ↪  std::pow(static_cast<double>(u) , static_cast<double>(j)) *
        ↪  std::pow(static_cast<double>(d), static_cast<double>(N-j))
        ↪  , static_cast<commonType>(0));
    }

    for (int i {}; i < N ; ++i)
    {
        std::vector<commonType> V_new (N-i);
        for (int k {}; k < N-i; ++k)
        {
            V_new[k] = p * V_current[k+1] + (1-p) * V_current[k];
        }
        std::swap(V_current, V_new);
    }

    return V_current[0] * std::exp(-data.rate * data.maturity);
}
```

Listing 3.2: Black-Scholes-Merton Put using Monte Carlo Simulations

This is exactly the same as the European Binomial Tree in Listing 3.1 except in line 20 it uses Equation 3.11 instead of Equation 3.10.

3.5 Notes on Binomial Tree without Vectors

For this section, we will again be using the GBM Assumption with the same structure as in Section 3.1.

3.5.1 Space Complexity of the Binomial Tree

This subsubsection will refer Listing 3.1
In line 17, we defined an array $\mathrm{V}_{\mathrm{current}}$ of length $N+1$. In line 25 we defined an array $\mathrm{V}_{\mathrm{new}}$ of length $N-i$ where $0 \leq i \leq N-1$. This array has the maximum length when $i = 0$ with length N

The total amount of memory the Binomial Tree occupies at a given time is:

$$\begin{aligned} \mathrm{Memory}_{\max} &\approxeq (N+1+N)\,\mathrm{sizeof(commonType)} \\ \mathrm{Memory}_{\max} &\approxeq (2N+1)\,\mathrm{sizeof(commonType)} \end{aligned} \tag{3.14}$$

Implementing this **Big-O Notation**

$$\begin{aligned} &\mathcal{O}(\mathrm{Memory}_{\max}) \\ &\mathcal{O}((2N+1)\,\mathrm{sizeof(commonType)}) \end{aligned}$$

Since only the highest degree terms are included in the Space-Complexity expression, and coefficients are removed, the space complexity of the Binomial Tree is:

$$\boxed{\mathcal{O}(N)} \tag{3.15}$$

3.5.2 Direct Formula with no Recursion

The direct formula for a call option is:

$$C = e^{-rT} \sum_{j=0}^{N} \binom{N}{j} p^j (1-p)^{N-j} \max(S_{0,0} u^j d^{N-j} - K, 0). \tag{3.16}$$

where:

$$\binom{n}{k} = \frac{1}{(n+1)B(n-k+1, k+1)} \tag{3.17}$$

```cpp
#ifndef FUNCTIONS
#define FUNCTIONS

#include <cmath>
inline double binomial(int n, int k)
{
    return 1/((n+1)*std::beta(n-k+1,k+1));
}

#endif
```

Listing 3.3: Binomial Factor using Beta Function

In C++ Equation 3.16 is implemented like this:

```cpp
#include <cmath>
#include <type_traits>
#include <algorithm>
#include "../Headers/functions.hpp"
#include "../Headers/black_scholes_struct.hpp"

template <typename S_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
auto binomialTreeBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
↪  data, const int N) -> decltype(data.spot)
{
    using commonType = decltype(data.spot);
    auto sqrt_deltat {std::sqrt(data.maturity /
    ↪  static_cast<commonType>(N))};

    auto u {std::exp(data.volatility * sqrt_deltat)};
    auto d {std::exp(-data.volatility * sqrt_deltat)};
    auto p {(std::exp(data.rate * data.maturity / N) - d) / (u -d)};

    commonType sum {};
    for (int j {}; j <= N; ++j)
    {
        sum += binomial(N,j) * std::pow(p, j) * std::pow(1-p, N-j) *
        ↪  std::max(data.spot * std::pow(u, j) * std::pow(d, N-j) -
        ↪  data.strike , static_cast<commonType>(0));
    }
    return sum * std::exp(-data.rate * data.maturity);
}
```

Listing 3.4: Binomial Tree for Black Scholes Call Options without Vectors

This code is almost the same as the normal Binomial Tree, but instead of creating two vectors, it uses an variable **sum** of type commonType. From line 18 it evaluates each value of the sum in Equation 3.16. for each value **j** that is in the range $0 \leq j \leq N$ and added to the sum, where finally in line 22 this final value is discounted.
For put options, the equation is as below:

$$P = e^{-rT} \sum_{j=0}^{N} \binom{N}{j} p^j (1-p)^{N-j} \max(K - S_{0,0} u^j d^{N-j}, 0). \tag{3.18}$$

```
1   #include <cmath>
2   #include <type_traits>
3   #include <algorithm>
4   #include "../Headers/functions.hpp"
5   #include "../Headers/black_scholes_struct.hpp"
6
7   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
8   auto binomialTreeBSPut(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
    ↪  data, const int N) -> decltype(data.spot)
9   {
10      using commonType = decltype(data.spot);
11      auto sqrt_deltat {std::sqrt(data.maturity /
        ↪  static_cast<commonType>(N))};
12
13      auto u {std::exp(data.volatility * sqrt_deltat)};
14      auto d {std::exp(-data.volatility * sqrt_deltat)};
15      auto p {(std::exp(data.rate * data.maturity / N) - d) / (u -d)};
16
17      commonType sum {};
18      for (int j {}; j <= N; ++j)
19      {
20          sum += binomial(N,j) * std::pow(p, j) * std::pow(1-p, N-j) *
            ↪  std::max(data.strike - data.spot * std::pow(u, j) * std::pow(d,
            ↪  N-j), static_cast<commonType>(0));
21      }
22      return sum * std::exp(-data.rate * data.maturity);
23  }
24
```

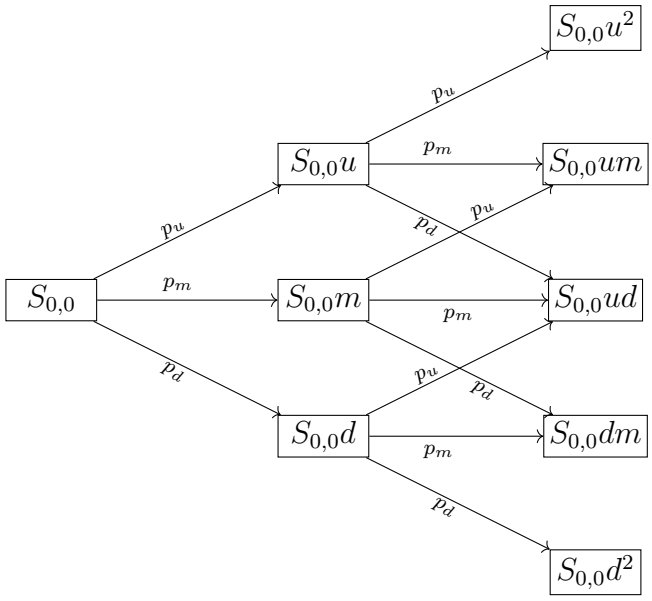
Listing 3.5: Binomial Tree for Black Scholes Put Options without Vectors

The only thing that changed was line 20 to represent Equation 3.18 instead of Equation 3.16. The reason I did not use both of these equation in the end is since it results in an **overflow error** after $N = 1030$

4 Trinomial Tree

Here is the file on my **GitHub**

4.1 Deriving the Risk-Neutral Probabilities



each step $= 2\Delta t$

Figure 3: Trinomial Tree for the underlying asset price

[6]

A step in the trinomial tree is basically just 2 binomial tree steps combined together. Let u_{tri}, d_{tri} and m represent the up, down and middle steps for the **trinomial** tree respectively. We will also define u_{bi} and d_{bi} as the up and down steps for the **binomial** tree
An up step in the trinomial tree is equivalent to two consecutive up steps in the binomial tree.

$$u_{tri} = u_{bi}^2 \tag{4.1}$$

Substituting $u_{bi} = e^{\sigma\sqrt{\Delta t}}$ into the previous equation we get:

$$\boxed{u_{tri} = e^{2\sigma\sqrt{\Delta t}}} \tag{4.2}$$

[6] A down step in the trinomial tree is equivalent to two consecutive down steps in the binomial tree.

$$d_{tri} = d_{bi}^2 \tag{4.3}$$

Substituting $d_{bi} = e^{-\sigma\sqrt{\Delta t}}$ into the previous equation we get:

$$\boxed{d_{tri} = e^{-2\sigma\sqrt{\Delta t}}} \tag{4.4}$$

[6] Finally a middle step in the trinomial tree is equivalent to an up and down step in the binomial tree.

$$m = u_{bi}d_{bi} \tag{4.5}$$

Substituting $u_{bi} = e^{\sigma\sqrt{\Delta t}}$ and $d_{bi} = e^{-\sigma\sqrt{\Delta t}}$ into the previous equation we get:

$$\boxed{m = 1} \tag{4.6}$$

This same concept applies for the probabilities. Using p as the probability of a binomial tree going up, and $1 - p$ as the probability of a binomial tree going down we have:

$$p_u = p^2 \tag{4.7}$$

Substituting $p = \frac{e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}}$ into the equation we get:

$$\boxed{p_u = (\frac{e^{r\Delta t} - e^{-\sigma\sqrt{\Delta t}}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}})^2} \tag{4.8}$$

[6]

$$p_d = (1 - p)^2 \tag{4.9}$$

Substituting p again we get:

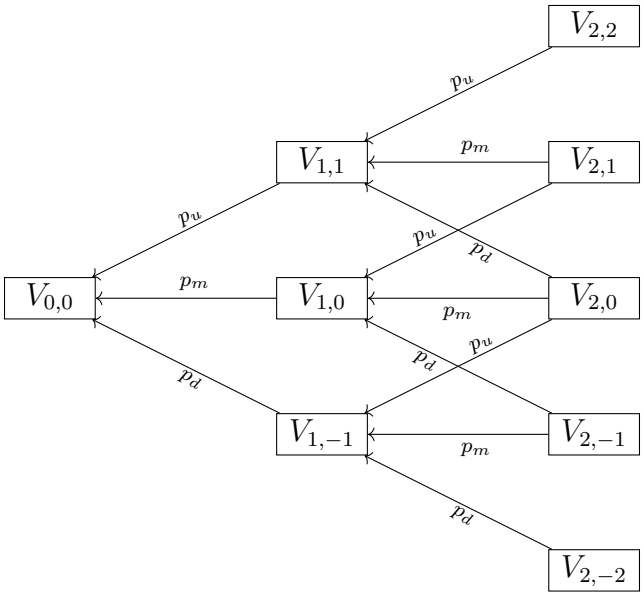
$$\boxed{p_d = (\frac{e^{-\sigma\sqrt{\Delta t}} - e^{r\Delta t}}{e^{\sigma\sqrt{\Delta t}} - e^{-\sigma\sqrt{\Delta t}}})^2} \tag{4.10}$$

[6] Finally since all of the probabilities must add up to 1 we get the expression for p_m to be:

$$\boxed{p_m = 1 - p_u - p_d} \tag{4.11}$$

[6] Using ta different definition from the Binomial Trees where $S_{i,j}$ represents the price of the stock at step i with j vertical position

$$S_{i,j} = u^j S_{0,0} \tag{4.12}$$



each step $= 2\Delta t$

Figure 4: Reverse Trinomial Tree to find Payoffs

[6] The expected value of a payoff $V_{i,j}$ is given similar to the binomial tree except now there is a middle term included.

$$V_{i,j} = e^{-r\Delta t}(p_u V_{i+1,j+1} + p_m V_{i+1,j} + p_d V_{i+1,j-1}) \tag{4.13}$$

In order to calculate the terminal payoffs for call options we use Equation 4.12. The size of the trinomial tree is n

$$V_{n,k} = \max(u^k S_{0,0} - K, 0) \tag{4.14}$$

where these values are computed for $-n \leq j \leq n$

4.2 Implementation of the Trinomial Tree for BS Call Options

```
1   #include <cmath>
2   #include <vector>
3   #include <type_traits>
4   #include <algorithm>
5   #include "../Headers/black_scholes_struct.hpp"
6
7   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
8   auto trinomialTreeBSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
    ↪  data, const int N) -> decltype(data.spot)
9   {
10      using commonType = decltype(data.spot);
11
12      auto sqrt_2deltat {std::sqrt(2 * data.maturity /
        ↪  static_cast<commonType>(N))};
13
14      auto u {std::exp(data.volatility * sqrt_2deltat)};
15      auto d {std::exp(-data.volatility * sqrt_2deltat)};
16
17      auto p_u {std::pow((std::exp(data.rate * data.maturity / (2 * N)) -
        ↪  std::exp(-data.volatility * sqrt_2deltat /
        ↪  static_cast<commonType>(2) )) / ((std::exp(data.volatility *
        ↪  sqrt_2deltat / static_cast<commonType>(2))) -
        ↪  std::exp(-data.volatility * sqrt_2deltat /
        ↪  static_cast<commonType>(2))), static_cast<commonType>(2))};
18      auto p_d {std::pow((std::exp(data.volatility * sqrt_2deltat /
        ↪  static_cast<commonType>(2)) - std::exp(data.rate * data.maturity /
        ↪  (2 * N))) / ((std::exp(data.volatility * sqrt_2deltat /
        ↪  static_cast<commonType>(2))) - std::exp(-data.volatility *
        ↪  sqrt_2deltat / static_cast<commonType>(2))),
        ↪  static_cast<commonType>(2))};
19      auto p_m {static_cast<commonType>(1) - p_u - p_d};
```

Listing 4.1: Trinomial Tree for Black Scholes Call Option (Part I)

This function called **trinomialTreeBSCall** takes in the struct parameter **OptionDataBS** as well as the number of iterations **N**.
Line 8 computes Equation 4.2. Similarly in line 9 it computes Equation 4.4.
In line 11 it uses Equation 4.8 in order to calculate the value of p_u just like in line 12 using Equation 4.10 to calculate p_d and finally in line 13 where it computes Equation 4.11 to get the value of p_m

```
14        std::vector<commonType> V_current (2 * N + 1);
15        for (int j {}; j < 2 * N + 1; ++j)
16        {
17            V_current[j] = std::max(data.spot * std::pow(static_cast<double>(u)
              ↪  , static_cast<double>(j- N)) - data.strike,
              ↪  static_cast<commonType>(0));
18        }
```

Listing 4.2: Trinomial Tree for Black Scholes Call Option (Part II)

Line 14 initialises a vector of size $2N + 1$ since $N - (-N) + 1 = 2N + 1$
Line 15 iterates through each of the values of the terminal payoff vector, where in line 17 the function computes Equation 4.14 for each value of j. In that Equation $k = j - N$ where j is the temporary variable defined in this function.

```
19        for (int i {}; i < N ; ++i)
20        {
21            std::vector<commonType> V_new (2 * N - 2 * i - 1);
22            for (int k {}; k < 2* N-2*i -1; ++k)
23            {
24                V_new[k] = p_u * V_current[k+2] + p_m * V_current[k+1] + p_d *
                  ↪  V_current[k];
25            }
26            std::swap(V_current, V_new);
27        }
28
29        return V_current[0] * std::exp(-data.rate * data.maturity);
30    }
```

Listing 4.3: Trinomial Tree for Black Scholes Call Option (Part III)

Then in Line 20 it starts the main backward recursion sequence where each time in line 22 we initialise a vector of size $2N - 2i - 1$ (where i is the temporary loop variable) and using this we iterate again in line 23 to assign each index value for $\mathrm{V}_{\mathrm{new}}$ using Equation 4.13. In Line 27 $\mathrm{V}_{\mathrm{current}}$ and $\mathrm{V}_{\mathrm{new}}$ are swapped and this process is done again with the new incremented value of i Finally when the length of $\mathrm{V}_{\mathrm{current}}$ is 1, that singular value is returned after being discounted in Line 30.

4.3 Time Complexity of Trinomial Trees

$$\begin{aligned} N_{iterations} &= 2N + 1 + \sum_{i=0}^{N-1}(2N - 2i - 1) \\ N_{iterations} &= N^2 + 2N + 1 \end{aligned} \tag{4.15}$$

Substituting this into the time complexity expression:

$$\mathcal{O}(N^2 + 2N + 1)$$

Since we take the term with the highest order and exclude constants, the final time complexity of the Binomial Tree is:

$$\boxed{\mathcal{O}(N^2)} \tag{4.16}$$

4.4 Implementation of the Trinomial Tree for BS Put Options

The only thing that really changes in the implementation for the Put option is the terminal payoff vector, shown with this equation:

$$V_{n,k} = \max(u^k S_{0,0} - K, 0) \tag{4.17}$$

```
14      std::vector<commonType> V_current (2 * N + 1);
15      for (int j {}; j < 2 * N + 1; ++j)
16      {
17          V_current[j] = std::max(data.strike data.spot *
            ↪  std::pow(static_cast<double>(u) , static_cast<double>(j- N)),
            ↪  static_cast<commonType>(0));
18      }
```

Listing 4.4: Trinomial Tree for Black Scholes Put Option (Part II)

The only thing that changed in this function (which is called **trinomialTreeBSPut**) is that in line 17 it sets the value of the initial vector $V_{current}$ using Equation 4.17 instead of Equation 4.14

5 Finite Difference Methods (FDM)

5.1 Time and Space Discretisation

Time is discretised by defining a uniform grid of N time steps between the current time-level index n as t_n and the maturity length T
For time to maturity τ we define:

$$\tau_n = T - t_n \tag{5.1}$$

The grid points are defined as for $0 \leq n \leq N$:

$$\tau_n = n\Delta\tau \tag{5.2}$$

The change in time to maturiy is:

$$\Delta\tau = \frac{T - t_0}{N} \tag{5.3}$$

Space, in this case the stock price domain where $[0, S_{max}]$ is also discretised into M grid points:

$$S_i = i\Delta S, \quad \Delta S = \frac{S_{max}}{M}, \quad i = 0, 1, ..., M \tag{5.4}$$

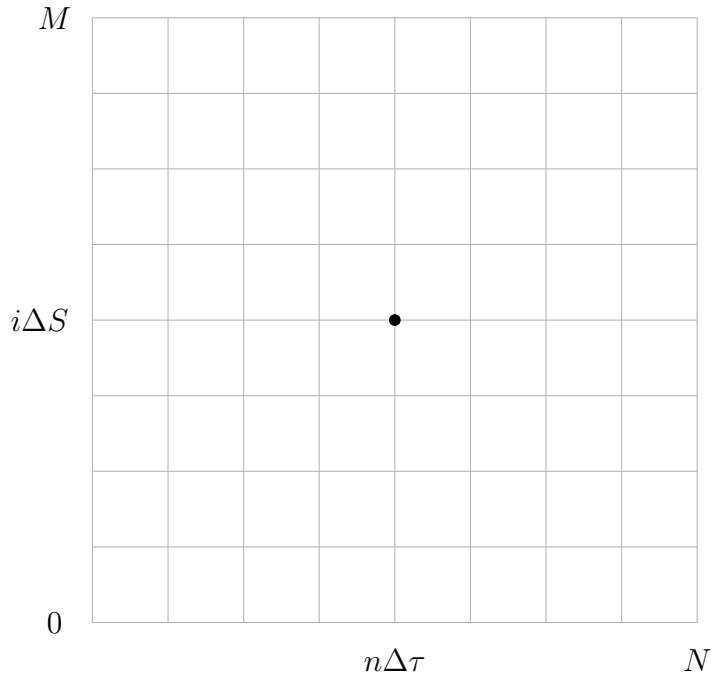


Figure 5: Mesh for the finite difference approximation

For the numerical solver, we can solve each case of (τ_n, S_i)

5.2 Finite Difference Schemes

The numerical approximation of an option value at time τ_n and at a stock level S_i is denoted as $V_i^n \approx V(t_n, S_i)$ To solve the Black Scholes PDE, finite difference schemes are used when calculating the space and time derivatives of each numerical solution. For **spatial derivatives**, the first derivative in space is:

$$V_S(\tau_n, S_i) \approx \frac{V_{i+1}^n - V_{i-1}^n}{2\Delta S} \tag{5.5}$$

The second derivative in space is:

$$V_{SS}(\tau_n, S_i) \approx \frac{V_{i+1}^n - 2V_i^n + V_{i-1}^n}{(\Delta S)^2} \tag{5.6}$$

The first derivative of the option's value with respect to the time to maturity is computed by taking the chain rule:

$$\frac{\partial V}{\partial t} = -\frac{\partial V}{\partial \tau} \tag{5.7}$$

The Black-Scholes Equation (which was derived in Equation 1.18) with the time-to-maturity is as the following

$$\frac{\partial V}{\partial \tau} = \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV \tag{5.8}$$

[7] We will approximate the time to maturity derivative using the **Crank-Nicolson** defined in the next subsection.

5.3 Crank-Nicolson Method

The Crank-Nicolson method is the intermediate between the explicit and implicit method. It takes the average of the spatial derivatives at n and $n+1$ We define an operator A_i^n for the spatial components of the Black-Scholes equation.

$$A_i^n = \frac{1}{2}\sigma^2 S^2 (\frac{V_{i+1}^n - 2V_i^n + V_{i-1}^n}{(\Delta S)^2}) + rS(\frac{V_{i+1}^n - V_{i-1}^n}{2\Delta S}) - rV_i^n$$

The Crank-Nicolson method says that the right hand side is an average spatial operator: $\frac{1}{2}[A_i^n + A_i^{n+1}]$. The left hand side is the difference of time derivatives $\frac{V_i^{n+1} - V_i^n}{\Delta \tau}$. Equating these sides, is the method:

$$\frac{V_i^{n+1} - V_i^n}{\Delta \tau} = \frac{1}{2}[A_i^n + A_i^{n+1}] \tag{5.9}$$

Define a linear operator $\mathcal{L}_h$ such that it satisfies $A_i^n := (\mathcal{L}_h V^n)_i$ This also means that the linear operator at each node i is:

$$(\mathcal{L}_h V^n)_i = \frac{1}{2}\sigma^2 S^2 (\frac{V_{i+1}^n - 2V_i^n + V_{i-1}^n}{(\Delta S)^2}) + rS(\frac{V_{i+1}^n - V_{i-1}^n}{2\Delta S}) - rV_i^n$$

Rearranging the equation above we get:

$$(\mathcal{L}_h V^n)_i = (\frac{1}{2}\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} - \frac{rS_i}{2\Delta S})(V_{i-1}) + (-\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} - r)(V_i) + (\frac{1}{2}\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} + \frac{rS_i}{2\Delta S})(V_{i+1})$$

Substituting $\alpha_i = \frac{1}{2}\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} - \frac{rS_i}{2\Delta S}, \quad \beta_i = -\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} - r, \quad \gamma_i = \frac{1}{2}\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} + \frac{rS_i}{2\Delta S}$
,

The linear operator is defined as a map of $\mathcal{L} : \mathbb{R}^{M-1} \to \mathbb{R}^{M-1}$ such that:

$$(\mathcal{L}_h V)_i = \alpha_i V_{i-1} + \beta_i V_i + \gamma_i V_{i+1} \tag{5.10}$$

Where the operator is given by the matrix:

$$\mathcal{L}_h = \begin{pmatrix} \beta_1 & \gamma_1 & 0 & \cdots & 0 \\ \alpha_2 & \beta_2 & \gamma_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \alpha_{M-2} & \beta_{M-2} & \gamma_{M-2} \\ 0 & \cdots & 0 & \alpha_{M-1} & \beta_{M-1} \end{pmatrix} \tag{5.11}$$

This way equation (3.9) becomes:

$$\frac{V_i^{n+1} - V_i^n}{\Delta\tau} = \frac{1}{2}[(\mathcal{L}_h V^n)_i + (\mathcal{L}_h V^{n+1})_i] \tag{5.12}$$

Rearranging the equation we get:

$$V_i^{n+1} - V_i^n = \frac{\Delta\tau}{2}[(\mathcal{L}_h V^n)_i + (\mathcal{L}_h V^{n+1})_i]$$

$$V_i^{n+1} - \frac{\Delta\tau}{2}(\mathcal{L}_h V^{n+1})_i = V_i^n + \frac{\Delta\tau}{2}(\mathcal{L}_h V^n)_i$$

$$V_i^{n+1}(I - \frac{\Delta\tau}{2}\mathcal{L}_h) = V_i^n(I + \frac{\Delta\tau}{2}\mathcal{L}_h)$$

Defining two matrices: $A := I - \frac{\Delta\tau}{2}\mathcal{L}_h$ and $B := I + \frac{\Delta\tau}{2}\mathcal{L}_h$ we get the equation:

$$AV^{n+1} = BV^n \tag{5.13}$$

where:

$$A = \begin{pmatrix} 1 - \frac{\Delta\tau}{2}\beta_1 & -\frac{\Delta\tau}{2}\gamma_1 & 0 & \cdots & 0 \\ -\frac{\Delta\tau}{2}\alpha_2 & 1 - \frac{\Delta\tau}{2}\beta_2 & -\frac{\Delta\tau}{2}\gamma_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -\frac{\Delta\tau}{2}\alpha_{M-2} & 1 - \frac{\Delta\tau}{2}\beta_{M-2} & -\frac{\Delta\tau}{2}\gamma_{M-2} \\ 0 & \cdots & 0 & -\frac{\Delta\tau}{2}\alpha_{M-1} & 1 - \frac{\Delta\tau}{2}\beta_{M-1} \end{pmatrix}$$

and:

$$B = \begin{pmatrix} 1+\frac{\Delta\tau}{2}\beta_1 & \frac{\Delta\tau}{2}\gamma_1 & 0 & \cdots & 0 \\ \frac{\Delta\tau}{2}\alpha_2 & 1+\frac{\Delta\tau}{2}\beta_2 & \frac{\Delta\tau}{2}\gamma_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \frac{\Delta\tau}{2}\alpha_{M-2} & 1+\frac{\Delta\tau}{2}\beta_{M-2} & \frac{\Delta\tau}{2}\gamma_{M-2} \\ 0 & \cdots & 0 & \frac{\Delta\tau}{2}\alpha_{M-1} & 1+\frac{\Delta\tau}{2}\beta_{M-1} \end{pmatrix}$$

In this tridiagonal matrix assembly, the goal is to solve for the vector V^{n+1} for each instance i. We will do this with the Thomas Algorithm . For a put call, if the stock is at 0, then selling the stock at that instance would be the best move since the stock cannot get any lower. This means that at that time, someone would make a payoff of K. This means that the discounted payoff is $Ke^{-r(\tau_n)}$ and that $V_0^n = Ke^{-r\tau_n)}$

5.4 Assembling the Tridiagonal Matrix

This section will be the code for $\alpha_i, \beta_i, \gamma_i$,

```
1   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
2   struct Coefficients
3   {
4       using commonType = std::common_type_t<S_t, K_t, r_t, sigma_t, T_t>;
5       std::vector <commonType> alphas{};
6       std::vector <commonType> betas{};
7       std::vector <commonType> gammas{};
8
9       explicit Coefficients(int M)
10      : alphas(M-1), betas(M-1), gammas(M-1)
11      {}
12  };
```

Listing 5.1: Struct for Coefficients

When an object with the type **Coefficients** is made, the program will initialise each member of Coefficients: that being **alphas** , **betas**, **gammas** with a vector of size $M-1$ using the constructor defined in Line 9.

$$\alpha_i = \frac{1}{2}\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} - \frac{rS_i}{2\Delta S} \tag{5.14}$$

$$\beta_i = -\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} - r \tag{5.15}$$

$$\gamma_i = \frac{1}{2}\sigma^2 S_i^2 \frac{1}{(\Delta S)^2} + \frac{rS_i}{2\Delta S} \tag{5.16}$$

```
1  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
   ↪  typename T_t>
2  Coefficients<S_t, K_t, r_t, sigma_t, T_t> generateCoefficientsBS(const
   ↪  OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>& data, const auto deltaS,
   ↪  const int M)
3  {
4      Coefficients<S_t, K_t, r_t, sigma_t, T_t> coeff{M};
5      using commonType = decltype(data.spot);
6      for (int i{0}; i < M-1; ++i)
7      {
8          commonType S_i {(i+1)*deltaS};
9          coeff.alphas[i] = 0.5 * (std::pow(data.volatility*S_i, 2.0)) /
           ↪  (std::pow(deltaS, 2.0)) - (data.rate*S_i) / (2*deltaS);
10         coeff.betas[i] = -(std::pow(data.volatility*S_i, 2.0)) /
           ↪  (std::pow(deltaS, 2.0)) - data.rate;
11         coeff.gammas[i] = 0.5 * (std::pow(data.volatility*S_i, 2.0)) /
           ↪  (std::pow(deltaS, 2.0)) + (data.rate*S_i) / (2*deltaS);
12     }
13     return coeff;
14 }
```

Listing 5.2: Generating the Coefficients

This function **generateCoefficients** uses the struct type **Coefficients** that was defined in the previous listing For this listing, Equation 5.14 was calculated in Line 9, Equation 5.15 was calculated in Line 10 and finally Equation 5.16 in Line 11. An important note it that since in C++ indexes start at 0, in Line 8 we use $S[i] = (i+1)\Delta S$ where the $+1$ accounts for the difference in indexes.

Next we will be creating the three diagonals for each A. The entry $A_{i,j}$ means the entry at row number i and column number j. Holding onto only the three diagonals instead of including the 0s save us tons of memory as the dimensions increase. The next listings will be for generating the diagonals of A

```
template <typename S_t, typename K_t, typename r_t, typename sigma_t,
↪  typename T_t>
struct Diagonals
{
    using commonType = std::common_type_t<S_t, K_t, r_t, sigma_t, T_t>;
    std::vector <commonType> A_lower {};
    std::vector <commonType> A_main {};
    std::vector <commonType> A_upper {};

    explicit Diagonals(int M)
    : A_lower(M-2), A_main(M-1), A_upper(M-2)
    {}
};
```

Listing 5.3: Struct for Diagonals

This struct is made to contain all of the members of Diagonals, which is $\mathrm{A}_{\mathrm{main}}$ for $A_{i,i}$, $\mathrm{A}_{\mathrm{lower}}$ for $A_{i,i+1}$ and $\mathrm{A}_{\mathrm{upper}}$ for $A_{i+1,i}$. The size of $\mathrm{A}_{\mathrm{main}}$ is 1 greater than that of $\mathrm{A}_{\mathrm{lower}}$ or $\mathrm{A}_{\mathrm{upper}}$ with a size of $M-1$ instead of $M-2$

$$A_{i,i} = 1 - \frac{\Delta\tau}{2}\beta_i, \qquad 1 \leq i \leq M-1 \tag{5.17}$$

$$A_{i,i+1} = -\frac{\Delta\tau}{2}\gamma_i, \qquad 1 \leq i \leq M-2 \tag{5.18}$$

$$A_{i+1,i} = -\frac{\Delta\tau}{2}\alpha_i, \qquad 1 \leq i \leq M-2 \tag{5.19}$$

```
1  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
   ↪  typename T_t>
2  Diagonals<S_t, K_t, r_t, sigma_t, T_t> generateDiagonalsBS(const
   ↪  Coefficients<S_t, K_t, r_t, sigma_t, T_t>& coeff, const auto deltaT,
   ↪  const int M)
3  {
4      Diagonals<S_t, K_t, r_t, sigma_t, T_t> diag {M};
5      for (int i {0}; i < M-2; ++i)
6      {
7          diag.A_main[i] = 1 - 0.5 * deltaT * coeff.betas[i];
8          diag.A_lower[i] = - 0.5 * deltaT * coeff.alphas[i+1];
9          diag.A_upper[i] = - 0.5 * deltaT * coeff.gammas[i];
10     }
11     diag.A_main[M-2] = 1 - 0.5 * deltaT * coeff.betas[M-2];
12     return diag;
13 }
```

Listing 5.4: Generating the Diagonals of A

This Listing computes Equation 5.17 in Line 7, 5.18 in Line 8 and finally 5.19 in Line 9. Additionally in Line 11, it computes the value of one more value of $\mathrm{A}_{\mathrm{main}}$ since $\mathrm{A}_{\mathrm{lower}}$ and $\mathrm{A}_{\mathrm{upper}}$ doesn't have a value at $i = M - 2$ which would have left an error during runtime.

5.5 Creating the RHS Vector for Calls

Since now we have programmed the elements in the equation. These sections will deal with actually solving the tridiagonal matrix. The Thomas Algorithm is used to solve equations like this where A and B are tridiagonal matrices.

$$AV^{n+1} = BV^n + b^n \tag{5.20}$$

We will begin with the base case. Over here we only include the interior nodes. The base case is:

$$V^0 = \begin{pmatrix} V_1^0 \\ V_2^0 \\ \vdots \\ V_{M-2}^0 \\ V_{M-1}^0 \end{pmatrix}$$

Using the fact that the payoff of an option at maturity to be at the time of purchase is $V = \max(S - K)$ since it does not go through discounting the base case is:

$$V^0 = \begin{pmatrix} \max(S_1 - K, 0) \\ \max(S_2 - K, 0) \\ \vdots \\ \max(S_{M-2} - K, 0) \\ \max(S_{M-1} - K, 0) \end{pmatrix}$$

Now substituting $S_i = i\Delta S$

$$V^0 = \begin{pmatrix} \max(\Delta S - K, 0) \\ \max(2\Delta S - K, 0) \\ \vdots \\ \max((M-2)\Delta S - K, 0) \\ \max((M-1)\Delta S - K, 0) \end{pmatrix} \tag{5.21}$$

Now we will use this case to find the values of $V^1, V^2, \ldots \quad, V^N$ First trying to find an equivalent expression to the right-hand side.

$$BV^n = \begin{pmatrix} 1 + \frac{\Delta\tau}{2}\beta_1 & \frac{\Delta\tau}{2}\gamma_1 & 0 & \cdots & 0 \\ \frac{\Delta\tau}{2}\alpha_2 & 1 + \frac{\Delta\tau}{2}\beta_2 & \frac{\Delta\tau}{2}\gamma_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \frac{\Delta\tau}{2}\alpha_{M-2} & 1 + \frac{\Delta\tau}{2}\beta_{M-2} & \frac{\Delta\tau}{2}\gamma_{M-2} \\ 0 & \cdots & 0 & \frac{\Delta\tau}{2}\alpha_{M-1} & 1 + \frac{\Delta\tau}{2}\beta_{M-1} \end{pmatrix} \begin{pmatrix} V_1^n \\ V_2^n \\ \vdots \\ V_{M-2}^n \\ V_{M-1}^n \end{pmatrix}$$

$$BV^n = \begin{pmatrix} (1 + \frac{\Delta\tau}{2}\beta_1)V_1^n + \frac{\Delta\tau}{2}\gamma_1 V_2^n \\ \frac{\Delta\tau}{2}\alpha_2 V_1^n + (1 + \frac{\Delta\tau}{2}\beta_2)V_2^n + \frac{\Delta\tau}{2}\gamma_2 V_3^n \\ \vdots \\ \frac{\Delta\tau}{2}\alpha_{M-2}V_{M-3}^n + (1 + \frac{\Delta\tau}{2}\beta_{M-2})V_{M-2}^n + \frac{\Delta\tau}{2}\gamma_{M-2}V_{M-1}^n \\ \frac{\Delta\tau}{2}\alpha_{M-1}V_{M-2}^n + (1 + \frac{\Delta\tau}{2}\beta_{M-1})V_{M-1}^n \end{pmatrix} \tag{5.22}$$

However in this expression we only take into consideration the interior values. It is important to note that this term only consists of the interior points, there are also boundary points that are known. On the top entry we are missing $\frac{\Delta\tau}{2}\alpha_1 V_0^n$ and on the bottom entry we are missing $\frac{\Delta\tau}{2}\gamma_{M-1}V_M^n$. The value of V_0^n is as shown:

$$V_0^n = \lim_{S \to 0^+} C(\tau, S)$$

Substituting Equation 1.19 with $\tau = T - t$ we get:

$$V_0^n = \lim_{S \to 0}(S\Phi(d_1) - Ke^{-r\tau}\Phi(d_2))$$

From Equation 1.19 we see that $\lim_{S \to 0^+} d_1 = -\infty$ and hence $\lim_{S \to 0^+} d_2 = -\infty$. Using the properties of cumulative distribution functions, we use $\lim_{S \to 0^+} \Phi(d_1) = 0$ and $\lim_{S \to 0^+} \Phi(d_2) = 0$ This gives us the final value of the beginning of the base case vector which is:

$$\boxed{V_0^n = 0} \tag{5.23}$$

Next up for our final vector we will use the limit as the stock goes to infinity.

$$V_M^n = \lim_{S \to +\infty} C(\tau, S)$$
$$V_M^n = \lim_{S \to +\infty} (S\Phi(d_1) - Ke^{-r\tau)}\Phi(d_2)$$

Since $\lim_{S \to \infty} d_1 = \infty$ and hence $\lim_{S \to +\infty} d_2 = \infty$. For the cumulative distribution functions $\lim_{S \to +\infty} \Phi(d_1) = 1$ and $\lim_{S \to +\infty} \Phi(d_2) = 1$ Now this limit does diverge to ∞, however the asymptotic behaviour of this option price is:

$$V_M^n \sim S - Ke^{-r\tau} \quad S \to \infty \tag{5.24}$$

We define S_{max} as the highest value of S we give in the finite difference grid. This makes the whole base case vector V^0 equal to:

$$\boxed{V_M^n = S_{max} - Ke^{-r\tau}} \tag{5.25}$$

$$V^0 = \begin{pmatrix} 0 \\ \max(\Delta S - K, 0) \\ \vdots \\ \max((M-1)\Delta S - K, 0) \\ S_{max} - Ke^{-r\tau} \end{pmatrix} \tag{5.26}$$

$$V_i^{n+1} - V_i^n = \frac{\Delta\tau}{2}[(\mathcal{L}_h V^n)_i + (\mathcal{L}_h V^{n+1})_i]$$
$$V_i^{n+1} - V_i^n = \frac{\Delta\tau}{2}[(\alpha_i V_{i-1}^n + \beta_i V_i^n + \gamma_i V_{i+1}^n)_i + (\alpha_i V_{i-1}^{n+1} + \beta_i V_i^{n+1} + \gamma_i V_{i+1}^{n+1})]$$

First we substitute the value when $i = 1$ to represent the lower boundary

$$V_1^{n+1} - V_1^n = \frac{\Delta\tau}{2}[(\alpha_1 V_0^n + \beta_i V_1^n + \gamma_1 V_2^n)_1 + (\alpha_1 V_0^{n+1} + \beta_1 V_1^{n+1} + \gamma_1 V_2^{n+1})]$$

All the $n+1$ terms are unknown except for V_0^{n+1} Moving all of the unknown $n+1$ terms to the left we get:

$$V_1^{n+1} - \frac{\Delta\tau}{2}(\beta_1 V_1^{n+1} + \gamma_1 V_2^{n+1}) = V_1^n + \frac{\Delta\tau}{2}(\alpha_1 V_0^n + \beta_i V_1^n + \gamma_1 V_2^n + \alpha_1 V_0^{n+1})$$
$$AV_1^{n+1} = BV_1^n + b_1^n$$

where $b_1^n = \frac{\Delta\tau}{2}\alpha_1 V_0^{n+1}$ which is also equal to 0.
Next up for the upper boundary we will substitute $i = M-1$ into this equation to get:

$$V_{M-1}^{n+1} - V_{M-1}^n = \frac{\Delta\tau}{2}[(\alpha_{M-1}V_{M-2}^n + \beta_i V_{M-1}^n + \gamma_{M-1}V_M^n)_{M-1} + (\alpha_{M-1}V_{M-2}^{n+1} + \beta_{M-1}V_{M-1}^{n+1} + \gamma_{M-1}V_M^{n+1})]$$

All of these terms are unknown except for $\gamma_{M-1}V_M^{n+1}$. Moving all of the unknown $n+1$ terms to the left we get:

$$V_{M-1}^{n+1} - \frac{\Delta\tau}{2}(\alpha_{M-1}V_{M-2}^{n+1} + \beta_{M-1}V_{M-1}^{n+1}) = V_{M-1}^n + \frac{\Delta\tau}{2}(\alpha_{M-1}V_{M-2}^n + \beta_i V_{M-1}^n + \gamma_{M-1}V_M^n + \gamma_{M-1}V_M^{n+1})$$
$$AV_{M-1}^{n+1} = BV_{M-1}^n + b_{M-1}^n$$

where $b_{M-1}^n = \frac{\Delta\tau}{2}\gamma_{M-1}V_M^{n+1}$ which is also equal to $\frac{\Delta\tau}{2}\gamma_{M-1}(S_{max} - Ke^{-r(n+1)\tau})$
Including both of the correction terms our RHS term is equal to:

$$RHS = \begin{pmatrix} \frac{\Delta\tau}{2}\alpha_1 V_0^n + (1 + \frac{\Delta\tau}{2}\beta_1)V_1^n + \frac{\Delta\tau}{2}\gamma_1 V_2^n \\ \frac{\Delta\tau}{2}\alpha_2 V_1^n + (1 + \frac{\Delta\tau}{2}\beta_2)V_2^n + \frac{\Delta\tau}{2}\gamma_2 V_3^n \\ \vdots \\ \frac{\Delta\tau}{2}\alpha_{M-2}V_{M-3}^n + (1 + \frac{\Delta\tau}{2}\beta_{M-2})V_{M-2}^n + \frac{\Delta\tau}{2}\gamma_{M-2}V_{M-1}^n \\ \frac{\Delta\tau}{2}\alpha_{M-1}V_{M-2}^n + (1 + \frac{\Delta\tau}{2}\beta_{M-1})V_{M-1}^n + \frac{\Delta\tau}{2}\gamma_{M-1}V_M^n \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ \frac{\Delta\tau}{2}\gamma_{M-1}(S_{\max} - Ke^{-r(n+1)\Delta\tau}) \end{pmatrix} \tag{5.27}$$

Next up is programming this vector

```
1   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪ typename T_t>
2   auto generateRHSCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>& data,
    ↪ Coefficients<S_t, K_t, r_t, sigma_t, T_t>& coeff,  auto V, const int n,
    ↪ const int M, const auto deltaT, const auto Smax)
3   {
4       using commonType = decltype(data.spot);
5       V[M] = Smax - data.strike *
        ↪ std::exp(-data.rate*static_cast<commonType>(n)*deltaT);
6       V[0] = 0;
7       std::vector<commonType> RHS (M-1);
8       for (int i{}; i < M-1; ++i)
9       {
10          RHS[i] = static_cast<commonType>(0.5)*deltaT*coeff.alphas[i]*V[i] +
            ↪ (static_cast<commonType>(1) +
            ↪ static_cast<commonType>(0.5)*deltaT*coeff.betas[i])*V[i+1] +
            ↪ static_cast<commonType>(0.5)*deltaT * coeff.gammas[i]*V[i+2];
11      }
12      RHS[M - 2] += (static_cast<commonType>(0.5) * deltaT * coeff.gammas[M -
        ↪ 2]) * (Smax - data.strike * std::exp(-data.rate *
        ↪ (static_cast<commonType>(n) + static_cast<commonType>(1)) *
        ↪ deltaT));
13      return RHS;
14  }
```

Listing 5.5: Generating the RHS Vector for Call Options

This function **generateRHSCall** will return a **std::vector** with element types of **commonType**. This function takes in the **OptionDataBS** and **Coefficients** structs as well as the vector V^n , the value of n, the value of M, the value of $\Delta\tau$ and finally the value of $S_{\max}$

This function first initialises a vector called **RHS** in Line 7 with a size of $M-1$ elements. This vector is then iterated for each index i where equation 5.27 is computed for each value in the vector.

Finally in Line 12, the correction term $\frac{\Delta\tau}{2}\gamma_{M-1}(S_{\max} - Ke^{-r(n+1)\tau})$ is added onto the value of this original vector, giving us the final vector and returning it in Line 13.

5.6 Creating the RHS Vector for Puts

This section is similar to the previous one except it uses the payoffs for a **put option** instead of a call option Again we will begin with the base case. The base case is:

$$V^0 = \begin{pmatrix} V_1^0 \\ V_2^0 \\ \vdots \\ V_{M-2}^0 \\ V_{M-1}^0 \end{pmatrix}$$

Using the fact that the payoff of an option at maturity to be at the time of purchase is $V = \max(K - S)$ since it does not go through discounting the base case is:

$$V^0 = \begin{pmatrix} \max(K - S_1, 0) \\ \max(K - S_2, 0) \\ \vdots \\ \max(K - S_{M-2}, 0) \\ \max(K - S_{M-1}, 0) \end{pmatrix}$$

Now substituting $S_i = i\Delta S$

$$V^0 = \begin{pmatrix} \max(K - \Delta S, 0) \\ \max(K - 2\Delta S, 0) \\ \vdots \\ \max(K - (M-2)\Delta S, 0) \\ \max(K - (M-1)\Delta S, 0) \end{pmatrix} \tag{5.28}$$

Now we will use this case to find the values of $V^1, V^2, \ldots \quad , V^N$ First trying to find an equivalent expression to the right-hand side.

$$BV^n = \begin{pmatrix} 1 + \frac{\Delta\tau}{2}\beta_1 & \frac{\Delta\tau}{2}\gamma_1 & 0 & \cdots & 0 \\ \frac{\Delta\tau}{2}\alpha_2 & 1 + \frac{\Delta\tau}{2}\beta_2 & \frac{\Delta\tau}{2}\gamma_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \frac{\Delta\tau}{2}\alpha_{M-2} & 1 + \frac{\Delta\tau}{2}\beta_{M-2} & \frac{\Delta\tau}{2}\gamma_{M-2} \\ 0 & \cdots & 0 & \frac{\Delta\tau}{2}\alpha_{M-1} & 1 + \frac{\Delta\tau}{2}\beta_{M-1} \end{pmatrix} \begin{pmatrix} V_1^n \\ V_2^n \\ \vdots \\ V_{M-2}^n \\ V_{M-1}^n \end{pmatrix}$$

The value of V_0^n is as shown:

$$V_0^n = \lim_{S \to 0^+} P(\tau, S)$$

Substituting Equation 1.19 with $\tau = T - t$ we get:

$$V_0^n = \lim_{S \to 0}(Ke^{-r\tau}\Phi(-d_2) - S\Phi(-d_1))$$

From Equation 1.19 we see that $\lim_{S \to 0^+} d_1 = -\infty$ and hence $\lim_{S \to 0^+} d_2 = -\infty$. For the cumulative distribution function Φ the limits are: $\lim_{S \to 0^+} \Phi(-d_1) = 1$ and $\lim_{S \to 0^+} \Phi(-d_2) = 1$ This gives us the final value of the beginning of the base case vector which is:

$$\boxed{V_0^n = Ke^{-r\tau}} \tag{5.29}$$

Next up for our final vector we will use the limit as the stock goes to infinity.

$$V_M^n = \lim_{S \to +\infty} C(\tau, S)$$
$$V_M^n = \lim_{S \to +\infty} (S\Phi(d_1) - Ke^{-r\tau}\Phi(d_2)$$

Since $\lim_{S \to \infty} d_1 = \infty$ and hence $\lim_{S \to +\infty} d_2 = \infty$. For the cumulative distribution functions $\lim_{S \to +\infty} \Phi(-d_1) = 0$ and $\lim_{S \to +\infty} \Phi(-d_2) = 0$ Thie gives us the value of V_M^n to be:

$$\boxed{V_M^n = 0} \tag{5.30}$$

Now just like for call options we will find the correction terms again

$$V_i^{n+1} - V_i^n = \frac{\Delta\tau}{2}[(\mathcal{L}_h V^n)_i + (\mathcal{L}_h V^{n+1})_i]$$
$$V_i^{n+1} - V_i^n = \frac{\Delta\tau}{2}[(\alpha_i V_{i-1}^n + \beta_i V_i^n + \gamma_i V_{i+1}^n)_i + (\alpha_i V_{i-1}^{n+1} + \beta_i V_i^{n+1} + \gamma_i V_{i+1}^{n+1})]$$

First we substitute the value when $i = 1$ to represent the lower boundary

$$V_1^{n+1} - V_1^n = \frac{\Delta\tau}{2}[(\alpha_1 V_0^n + \beta_i V_1^n + \gamma_1 V_2^n)_1 + (\alpha_1 V_0^{n+1} + \beta_1 V_1^{n+1} + \gamma_1 V_2^{n+1})]$$

All the $n+1$ terms are unknown except for V_0^{n+1} Moving all of the unknown $n+1$ terms to the left we get:

$$V_1^{n+1} - \frac{\Delta\tau}{2}(\beta_1 V_1^{n+1} + \gamma_1 V_2^{n+1}) = V_1^n + \frac{\Delta\tau}{2}(\alpha_1 V_0^n + \beta_i V_1^n + \gamma_1 V_2^n + \alpha_1 V_0^{n+1})$$
$$AV_1^{n+1} = BV_1^n + b_1^n$$

where $b_1^n = \frac{\Delta\tau}{2}\alpha_1 V_1^{n+1}$ which is also equal to $\frac{\Delta\tau}{2}\alpha_1 K e^{-r(n+1)\Delta\tau}$.
1
Next up for the upper boundary we will substitute $i = M-1$ into this equation to get:

$$V_{M-1}^{n+1} - V_{M-1}^n = \frac{\Delta\tau}{2}[(\alpha_{M-1}V_{M-2}^n + \beta_i V_{M-1}^n + \gamma_{M-1}V_M^n)_{M-1} + (\alpha_{M-1}V_{M-2}^{n+1} + \beta_{M-1}V_{M-1}^{n+1} + \gamma_{M-1}V_M^{n+1})]$$

All of these terms are unknown except for $\gamma_{M-1}V_M^{n+1}$. Moving all of the unknown $n+1$ terms to the left we get:

$$V_{M-1}^{n+1} - \frac{\Delta\tau}{2}(\alpha_{M-1}V_{M-2}^{n+1} + \beta_{M-1}V_{M-1}^{n+1}) = V_{M-1}^n + \frac{\Delta\tau}{2}(\alpha_{M-1}V_{M-2}^n + \beta_i V_{M-1}^n + \gamma_{M-1}V_M^n + \gamma_{M-1}V_M^{n+1})$$
$$AV_{M-1}^{n+1} = BV_{M-1}^n + b_{M-1}^n$$

where $b_{M-1}^n = \frac{\Delta\tau}{2}\gamma_{M-1}V_M^{n+1}$ which is also equal to 0 Including both of the correction terms our RHS term is equal to:

$$RHS = \begin{pmatrix} \frac{\Delta\tau}{2}\alpha_1 V_0^n + (1 + \frac{\Delta\tau}{2}\beta_1)V_1^n + \frac{\Delta\tau}{2}\gamma_1 V_2^n \\ \frac{\Delta\tau}{2}\alpha_2 V_1^n + (1 + \frac{\Delta\tau}{2}\beta_2)V_2^n + \frac{\Delta\tau}{2}\gamma_2 V_3^n \\ \vdots \\ \frac{\Delta\tau}{2}\alpha_{M-2}V_{M-3}^n + (1 + \frac{\Delta\tau}{2}\beta_{M-2})V_{M-2}^n + \frac{\Delta\tau}{2}\gamma_{M-2}V_{M-1}^n \\ \frac{\Delta\tau}{2}\alpha_{M-1}V_{M-2}^n + (1 + \frac{\Delta\tau}{2}\beta_{M-1})V_{M-1}^n + \frac{\Delta\tau}{2}\gamma_{M-1}V_M^n \end{pmatrix} + \begin{pmatrix} \frac{\Delta\tau}{2}\alpha_1 K e^{-r(n+1)\Delta\tau} \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix} \tag{5.31}$$

Next up is programming this vector in C++

```
1  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
   ↪ typename T_t>
2  auto generateRHSPut(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>& data,
   ↪ Coefficients<S_t, K_t, r_t, sigma_t, T_t>& coeff,  auto V, const int n,
   ↪ const int M, const auto deltaT, const auto Smax)
3  {
4      using commonType = decltype(data.spot);
5      V[0] = data.strike *
       ↪ std::exp(-data.rate*static_cast<commonType>(n)*deltaT);
6      V[M] = 0;
7      std::vector<commonType> RHS (M-1);
8      for (int i{0}; i < M-1; ++i)
9      {
10         RHS[i] = static_cast<commonType>(0.5)*deltaT*coeff.alphas[i] * V[i]
           ↪ + (static_cast<commonType>(1) +
           ↪ static_cast<commonType>(0.5)*deltaT*coeff.betas[i])*V[i+1] +
           ↪ static_cast<commonType>(0.5)*deltaT * coeff.gammas[i]*V[i+2];
11     }
12     RHS[0] += static_cast<commonType>(0.5) * deltaT * coeff.alphas[0] *
       ↪ (data.strike * std::exp(-data.rate * (static_cast<commonType>(n) +
       ↪ static_cast<commonType>(1)) * deltaT ));
13     return RHS;
14 }
```

Listing 5.6: Generating the RHS Vector for Put Options

This function **generateRHSPut** again returns a **std::vector** with element types of **commonType**. This function again takes in the same parametrs as the previous one and again initialises a vector called **RHS** In Line 7 with a size of $M-1$ elements. This vector is then iterated for each index i where equation 5.31 is computed for each value in the vector.
Finally in Line 12, the correction term $\frac{\Delta\tau}{2}\alpha_1 K e^{-r(n+1)\Delta\tau}$ is added onto the value of this original vector, giving us the final vector and returning it in Line 13.

5.7 Thomas Algorithm

Now we will solve the tridiagonal system with the Thomas Algorithm

$$\begin{pmatrix} A_{1,1} & A_{1,2} & 0 & \cdots & 0 \\ A_{2,1} & A_{2,2} & A_{2,3} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & A_{M-2,M-3} & A_{M-2,M-2} & A_{M-2,M-1} \\ 0 & \cdots & 0 & A_{M-1,M-2} & A_{M-1,M-1} \end{pmatrix} \begin{pmatrix} V_1^{n+1} \\ V_2^{n+1} \\ \vdots \\ V_{M-2}^{n+1} \\ V_{M-1}^{n+1} \end{pmatrix} = \begin{pmatrix} BV_1^n \\ BV_2^n \\ \vdots \\ BV_{M-2}^n \\ BV_{M-1}^n \end{pmatrix}$$

To make this look a bit cleaner, we will define $\rho_i = \frac{A_{i,i+1}}{A_{i,i}}, \quad \eta_i = \frac{A_{i,i+1}}{A_{i,i}}, \quad \mu_i = \frac{BV_i^n}{A_{i,i}}$

$$\begin{pmatrix} 1 & \rho_1 & 0 & \cdots & 0 \\ \eta_2 & 1 & \rho_2 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \eta_{M-2} & 1 & \rho_{M-2} \\ 0 & \cdots & 0 & \eta_{M-1,} & 1 \end{pmatrix} \begin{pmatrix} V_1^{n+1} \\ V_2^{n+1} \\ \vdots \\ V_{M-2}^{n+1} \\ V_{M-1}^{n+1} \end{pmatrix} = \begin{pmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_{M-2} \\ \mu_{M-1} \end{pmatrix}$$

From rows 1 and 2 we have:

$$V_1^{n+1} + \rho_1 V_2^{n+1} = \mu_1$$

$$\eta_2 V_1^{n+1} + V_2^{n+1} + \rho_2 V_3^{n+1} = \mu_2$$

Multiplying row 1 by η_2 we get:

$$\eta_2 V_1^{n+1} + \eta_2 \rho_1 V_2^{n+1} = \eta_2 \mu_1$$

$$\eta_2 V_1^{n+1} + V_2^{n+1} + \rho_2 V_3^{n+1} = \mu_2$$

Subtracting row 2 by row 1 to make the new row 2, we get:

$$(1 - \eta_2 \rho_1) V_2^{n+1} + \rho_2 V_3^{n+1} = \mu_2 - \eta_2 \mu_1$$

Dividing the row by $1 - \eta_2 \rho_1$ we get:

$$V_2^{n+1} + \frac{\rho_2 V_3^{n+1}}{(1 - \eta_2 \rho_1)} = \frac{\mu_2 - \eta_2 \mu_1}{(1 - \eta_2 \rho_1)}$$

Putting this back into the matrix the matrix system becomes:

$$\begin{pmatrix} 1 & \rho_1 & 0 & \cdots & 0 \\ 0 & 1 & \frac{\rho_2}{(1-\eta_2\rho_1)} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & \eta_{M-2} & 1 & \rho_{M-2} \\ 0 & \cdots 5 & 0 & \eta_{M-1,} & 1 \end{pmatrix} \begin{pmatrix} V_1^{n+1} \\ V_2^{n+1} \\ \vdots \\ V_{M-2}^{n+1} \\ V_{M-1}^{n+1} \end{pmatrix} = \begin{pmatrix} \mu_1 \\ \frac{\mu_2-\eta_2\mu_1}{(1-\eta_2\rho_1)} \\ \vdots \\ \mu_{M-2} \\ \mu_{M-1} \end{pmatrix}$$

We will define $\rho_2' = \frac{\rho_2}{(1-\eta_2\rho_1)}$ and $\mu_2' = \frac{\mu_2}{1-\eta_2\mu_1}(1-\eta_2\rho_1)$ and continue this cycle. Now generalising this for each row $2 \leq i \leq M-1$, we get:

$$\rho_i' = \frac{\rho_i}{(1-\eta_i\rho_{i-1}')}, \qquad \rho_1' = \rho_1 \tag{5.32}$$

And for the right hand vector:

$$\mu_i' = \frac{\mu_i - \eta_i\mu_{i-1}'}{(1-\eta_i\rho\prime_{i-1})} \tag{5.33}$$

The matrix at the end of this stage of iterations is:

$$\begin{pmatrix} 1 & \rho_1' & 0 & \cdots & 0 \\ 0 & 1 & \rho_2' & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & 0 & 1 & \rho_{M-2}' \\ 0 & \cdots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} V_1^{n+1} \\ V_2^{n+1} \\ \vdots \\ V_{M-2}^{n+1} \\ V_{M-1}^{n+1} \end{pmatrix} = \begin{pmatrix} \mu_1' \\ \mu_2' \\ \vdots \\ \mu_{M-2}' \\ \mu_{M-1}' \end{pmatrix} \tag{5.34}$$

The final step is to solve for the vector through a method of **backwards substitution** from the last row we are given that:

$$V_{M-1}^{n+1} = \mu_{M-1}' \tag{5.35}$$

Solving for the value on the second last row, we get:

$$V_{M-2}^{n+1} + \rho_{M-2}' V_{M-1}^{n+1} = \mu_{M-2}'$$

Generalising this for $1 \leq i \leq M-2$ we get:

$$V_i^{n+1} = \mu_i' - \rho_i' V_{i+1}^{n+1}, \qquad V_{M-1}^{n+1} = \mu_{M-1}' \tag{5.36}$$

[8]

```
1  template <typename S_t, typename K_t, typename r_t, typename sigma_t,
   ↪  typename T_t>
2  auto thomasAlgorithm(Diagonals<S_t, K_t, r_t, sigma_t, T_t>& diag, const
   ↪  auto& RHS, const int M)
3  {
4      using commonType = std::remove_reference_t<decltype(diag.A_upper[0])>;
5      std::vector<commonType> rho_p (M-2);
6      std::vector<commonType> mu_p (M-1);
7
8      rho_p[0] = diag.A_upper[0] / diag.A_main[0];
9      mu_p[0] = RHS[0] / diag.A_main[0];
10
11     for (int i {1}; i < M - 2; ++i)
12     {
13         mu_p[i] = (RHS[i] / diag.A_main[i] - diag.A_lower[i-1] /
           ↪  diag.A_main[i] * mu_p[i-1]) / (static_cast<commonType>(1) -
           ↪  diag.A_lower[i-1] / diag.A_main[i] * rho_p[i-1]);
14         rho_p[i] = diag.A_upper[i] / diag.A_main[i] / (1.0 -
           ↪  diag.A_lower[i-1] / diag.A_main[i] * rho_p[i-1]);
15     }
16     mu_p[M-2] = (RHS[M-2] / diag.A_main[M-2] - diag.A_lower[M-3] /
       ↪  diag.A_main[M-2] * mu_p[M-3]) / (static_cast<commonType>(1) -
       ↪  diag.A_lower[M-3] / diag.A_main[M-2] * rho_p[M-3]);
17
18     std::vector<commonType> Vn (M);
19     Vn[M-2] = mu_p[M-2];
20     for (int i{M-3}; i >= 0; --i)
21     {
22         Vn[i] = mu_p[i] - rho_p[i] * Vn[i+1];
23     }
24     return Vn;
25 }
```

Listing 5.7: Thomas Algorithm

The function **thomasAlgorithm** takes in the parameter using a reference to the **Diagonals** struct as well as a reference to the **RHS** vector and finally a final parameter **M**.
In Line 4 we declare the commonType we will use, by removing the reference on $A_{upper}[0]$ in order to give us a clean type to work with.
In Line 8 we compute the definition of ρ_1 and store it in the vector rho_p for index 0. Similarly with the definition of μ_1 in Line 9. In Line 11 we iterate from $i = 1$ to $i = M - 3$ and compute the formulae for ρ' and μ'.

Line 13 computes Equation 5.33 and Line 13 computes Equation 5.32 , with both using the base cases defined in Line 8 and 9 respectively. Finally we use the base case of V_{M-1}^{n-1} defined in Equation 5.35 which is computed in Line 19. Line 20 is the beginning of the **backward substitution** where Line 22 computes Equation 5.36. Finally in Line 24 this vector is returned.

5.8 FDM for Call Options

Now will be the part where we actually call these functions and use it to price. Calling the previous functions to price.

```
1   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪ typename T_t>
2   auto stepCrankNicolsonCall(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
    ↪ data, Coefficients<S_t, K_t, r_t, sigma_t, T_t>& coeff, Diagonals<S_t,
    ↪ K_t, r_t, sigma_t, T_t>& diag, const auto deltaT, const int M, const
    ↪ int n, const auto Smax, auto V)
3   {
4       using commonType = decltype(data.spot);
5       std::vector<commonType> RHS{generateRHSCall(data, coeff, V, n, M,
        ↪ deltaT, Smax)};
6       std::vector<commonType> V_next {thomasAlgorithm(diag, RHS, M)};
7       V_next[0] = 0;
8       V_next.push_back(Smax - data.strike *
        ↪ std::exp(-data.rate*deltaT*(static_cast<commonType>(n+1))));
9       return V_next;
10  }
```

Listing 5.8: Crank-Nicolson Step for Call Options

This function **stepCrankNicolsonCall** takes in all of the parameters neccesary to compute **RHS** and to compute the Thomas Algorithm. Whenever the parameters are expensive, we pass by reference instead of by value. In Line 5 we compute the **RHS** vector using the function defined in Listing 5.5. This vector is then used as an argument in Line 6 for the **thomasAlgorithm** function defined in Listing 5.7 with the return value being stored in a vector called $\mathrm{V_{next}}$. This vector is then given the boundary values (V_1 in Line 7 and V_M in Line 8) and then finally returned in Line 9.

The final thing that is needed before we can actually price the option, is to figure out what to even do with the vector V^N Since we are looking to price the option when the stock is at S_t we need to figure out what exactly is that index in this list. We use the formula:

$$j_{option} = \frac{S_t}{\Delta S} \tag{5.37}$$

But if there is no i_{option} that is an integer, we must interpolate this, using the formula:

$$V(S_t) \approx V_j + \frac{S_t - S_j}{S_{j+1} - S_j}(V_{j+1} - V_j) \tag{5.38}$$

where $i \leq i_{option} \leq i + 1$

The next thing is defining $S_{multiplier}$ which is an indicator on how much higher the stock is compared to the strike price

$$S_{\max} = S_{multiplier} \times \max(S_t, K) \tag{5.39}$$

Using $S_{multiplier} = 4$ is a safe bet that no stocks go significantly higher than the strike price K These parameters will be very useful when defining the final function implement all of the other functions in order to price the European Option contracts using Finite Difference Methods.

```cpp
template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    typename T_t>
auto finiteDifferencesCallBS(const OptionDataBS<S_t, K_t, r_t, sigma_t,
    T_t>& data, const int M, const int N, const double multiplier=4.0)
{
    using commonType = decltype(data.spot);
    commonType Smax {multiplier * std::max(data.spot,data.strike)};
    commonType deltaS {Smax / static_cast<commonType>(M)};
    commonType deltaT {data.maturity / static_cast<commonType>(N)};
    std::vector<commonType> S (M + 1);
    for (int i{}; i < M + 1; ++i)
    {
        S[i] = static_cast<commonType>(i) * deltaS;
    }

    std::vector<commonType> V (M + 1);
    for (int i{}; i < M + 1; ++i)
    {
        V[i] = std::max(S[i] - data.strike, static_cast<double>(0));
    }

    Coefficients coeff {generateCoefficientsBS(data, deltaS, M)};
    Diagonals diag {generateDiagonalsBS(coeff, deltaT, M)};

    for (int n{}; n < N; ++n)
    {
        auto V_new {stepCrankNicolsonCall(data, coeff, diag , deltaT, M, n,
            Smax, V)};
        std::swap(V_new, V);
    }
    int j {static_cast<int>(data.spot / deltaS)};
    j = std::max(static_cast<int>(0),
    static_cast<int>(std::min(M-1,static_cast<int>(j))));
    commonType price {V[j] + (data.spot - S[j]) * (V[j+1] - V[j]) / (S[j+1]
    - S[j])};
    return price;
}
```

Listing 5.9: Finite Difference Method for Call Options

Line 5 computes the value of S_{max} using Equation 5.39 and stores it as a variable of type **commonType**. From there it uses this value of S_{max} to calculate ΔS using Equation 5.3 in Line 6. In Line 7 it calculates ΔT using Equation 5.3. Line 14 initialises a vector **V** with size $M + 1$ This vector is then used to store the calculated terminal payoffs that were defined in Equation 5.21. After that in Line 20 it initialises a struct object **coeff** using the function defined in Listing 5.2 . Line 21 initialises another strcut object **diag** using the function defined in Listing 5.4 In Line 25 we initialise a new vector called V_{new} using the return value from teh **stepCrankNicolsonCall** function defined in Listing 5.8. In Line 26 we swap the values of V and V_{new}. This process is iterated while $n < N$. Finally in Line 28 we define the integer value of **j** using the formula defined in Equation 5.37. Finally this value is used to compute the price which is finally returned in Line 31.

5.9 FDM for Put Options

Now will be the part where we actually call these functions and use it to price. Calling the previous functions to price.

```
1   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪ typename T_t>
2   auto stepCrankNicolsonPut(const OptionDataBS<S_t, K_t, r_t, sigma_t, T_t>&
    ↪ data, Coefficients<S_t, K_t, r_t, sigma_t, T_t>& coeff, Diagonals<S_t,
    ↪ K_t, r_t, sigma_t, T_t>& diag, const auto deltaT, const int M, const
    ↪ int n, const auto Smax, auto V)
3   {
4       using commonType = decltype(data.spot);
5       std::vector<commonType> RHS{generateRHSPut(data, coeff, V, n, M,
        ↪ deltaT, Smax)};
6       std::vector<commonType> V_next {thomasAlgorithm(diag, RHS, M)};
7       V_next[0] = data.strike *
        ↪ std::exp(-data.rate*deltaT*(static_cast<commonType>(n+1)));
8       V_next.push_back(0);
9       return V_next;
10  }
```

Listing 5.10: Crank-Nicolson Step for Call Options

The only thing different in this function **stepCrankNicolsonPut** compared to that defined in Listing 5.8 is that the boundary values of V_{next} is different for index 0. Now to implement the entire function that will return the price.

```
1   template <typename S_t, typename K_t, typename r_t, typename sigma_t,
    ↪  typename T_t>
2   auto finiteDifferencesPutBS(const OptionDataBS<S_t, K_t, r_t, sigma_t,
    ↪  T_t>& data, const int M, const int N, const double multiplier=4.0)
3   {
4       using commonType = decltype(data.spot);
5       commonType Smax {multiplier * std::max(data.spot,data.strike)};
6       commonType deltaS {Smax / static_cast<commonType>(M)};
7       commonType deltaT {data.maturity / static_cast<commonType>(N)};
8       std::vector<commonType> S (M + 1);
9       for (int i{}; i < M + 1; ++i)
10      {
11          S[i] = static_cast<commonType>(i) * deltaS;
12      }
13
14      std::vector<commonType> V (M + 1);
15      for (int i{}; i < M + 1; ++i)
16      {
17          V[i] = std::max(data.strike - S[i], static_cast<double>(0));
18      }
19
20      Coefficients coeff {generateCoefficientsBS(data, deltaS, M)};
21      Diagonals diag {generateDiagonalsBS(coeff, deltaT, M)};
22
23      for (int n{}; n < N; ++n)
24      {
25          auto V_new {stepCrankNicolsonPut(data, coeff, diag , deltaT, M, n,
            ↪  Smax, V)};
26          std::swap(V_new, V);
27      }
28      int j {static_cast<int>(data.spot / deltaS)};
29      j = std::max(static_cast<int>(0),
        ↪  static_cast<int>(std::min(M-1,static_cast<int>(j))));
30      double price {V[j] + (data.spot - S[j]) * (V[j+1] - V[j]) / (S[j+1] -
        ↪  S[j])};
31      return price;
32  }
```

Listing 5.11: Finite Difference Method for Put Options

The only things that changed in this function **finteDifferencesPutBS** is that the base case in Line 17 is different and used to represent Equation 5.28 instead of Equation 5.21. The only other thing that changes is that Line 25 uses the function **stepCrankNicolsonPut** defined in Listing 5.10

References

[1] Shreve, S. (2004). *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer Science & Business Media.

[2] Lalley, S. P. (n.d.). *Brownian motion*. https://galton.uchicago.edu/~lalley/Courses/313/BrownianMotionCurrent.pdf

[3] Haugh, M. (2016). *The Black-Scholes model* (IEOR E4706: Foundations of Financial Engineering lecture notes). Columbia University. https://www.columbia.edu/~mh2078/FoundationsFE/BlackScholes.pdf

[4] Chaudhary, A. (2025). *Solving Black-Scholes equations by Monte Carlo methods* (Mathematical Finance exercise sheet). University of Tübingen. https://na.uni-tuebingen.de/ex/math_finance_ss25/Exercise_5th.pdf

[5] Ocone, D. (2006). *Binomial trees and Black-Scholes* (Financial Mathematics 640:495 lecture notes). Rutgers University. https://sites.math.rutgers.edu/courses/495/495f06/lect21notes.pdf

[6] Josheski, D., & Apostolov, M. (2020). *A review of the binomial and trinomial models for option pricing and their convergence to the Black-Scholes model determined option prices*. Econometrics, 24(2), 53–85. https://doi.org/10.15611/eada.2020.2.05

[7] Fernandes, M. M. (2009). *Finite differences schemes for pricing of European and American options* (Extended abstract). IST, Technical University of Lisbon. https://fenix.tecnico.ulisboa.pt/downloadFile/395139424085/Extended%20Abstract.pdf

[8] Lee, W. T. (n.d.). *Tridiagonal matrices: Thomas algorithm* (MS6021: Scientific Computation lecture notes). University of Limerick. https://sites.nd.edu/gfu/files/2019/10/ms6021_thomas.pdf